
Documentação de Projeto

para o sistema

MecaFlow

Versão 1.0

Projeto de sistema elaborado pelo aluno Lucas José Lopes Ferreira
como parte da disciplina **Projeto de Software**.

10 de junho de 2026



Tabela de Conteúdo

(No Word, clique sobre o sumário e pressione F9 para atualizar os números de página.)

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Lucas José Lopes Ferreira	10/06/2026	Versão inicial do documento	1.0

1. Introdução

Este documento reúne os modelos de usuário, de projeto e de dados do MecaFlow, um sistema de gestão para oficinas mecânicas de pequeno e médio porte. O MecaFlow cobre o ciclo completo do reparo: o atendente cadastra cliente e veículo, agenda o serviço, emite o orçamento e, com a aprovação do cliente, abre a ordem de serviço; o mecânico executa o reparo e lança os serviços e as peças consumidas; o sistema debita o estoque a cada peça lançada; o pagamento encerra o ciclo e o atendimento fica registrado no histórico do veículo.

O escopo desta etapa cobre a modelagem e a arquitetura do sistema. O documento não trata da implementação do código. Todos os diagramas foram produzidos em PlantUML; os arquivos-fonte (.puml) e as imagens renderizadas estão versionados no repositório do projeto, na pasta docs/diagrams.

O documento segue esta organização: a Seção 2 descreve os atores, o modelo de casos de uso, as histórias de usuário, os diagramas de sequência do sistema e os contratos de operação; a Seção 3 apresenta os modelos de projeto (arquitetura, componentes, implantação, classes, sequência, comunicação e estados); a Seção 4 apresenta o modelo de dados.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

Três atores interagem com o MecaFlow. A tabela abaixo descreve o papel e as responsabilidades de cada um.

Ator	Descrição e responsabilidades
Administrador	Dono ou gerente da oficina. Gerencia os usuários do sistema, o catálogo de serviços e os fornecedores, e consulta os relatórios gerenciais de faturamento e produtividade (UC-02, UC-03, UC-04, UC-18).
Atendente	Responsável pela recepção. Cadastra clientes, veículos e peças, agenda serviços, emite orçamentos, registra a decisão do cliente, abre ordens de serviço, atribui mecânicos e registra pagamentos (UC-05 a UC-12, UC-16, UC-17).
Mecânico	Executa o reparo. Atualiza o status da ordem de serviço, lança os serviços e as peças consumidas, conclui a OS e consulta o histórico do veículo (UC-13, UC-14, UC-15, UC-17).

2.2 Modelo de Casos de Uso e Histórias de Usuários

A Figura 1 apresenta o diagrama de casos de uso do MecaFlow. Cada caso de uso recebe um identificador (UC-01 a UC-22) usado como referência no restante deste documento. Os relacionamentos de inclusão marcam comportamento obrigatório e reutilizado: UC-09 (Emitir orçamento) e UC-15 (Concluir ordem de serviço) incluem UC-19 (Calcular valor total). Os relacionamentos de extensão marcam comportamento condicional, com as condições listadas na legenda do diagrama: a baixa de estoque (UC-20) estende o lançamento de itens (UC-14), o alerta de estoque mínimo (UC-21) estende a baixa, e o desconto (UC-22) estende o pagamento (UC-16). A pré-condição de UC-11 (orçamento aprovado) aparece como nota, e não como relacionamento.

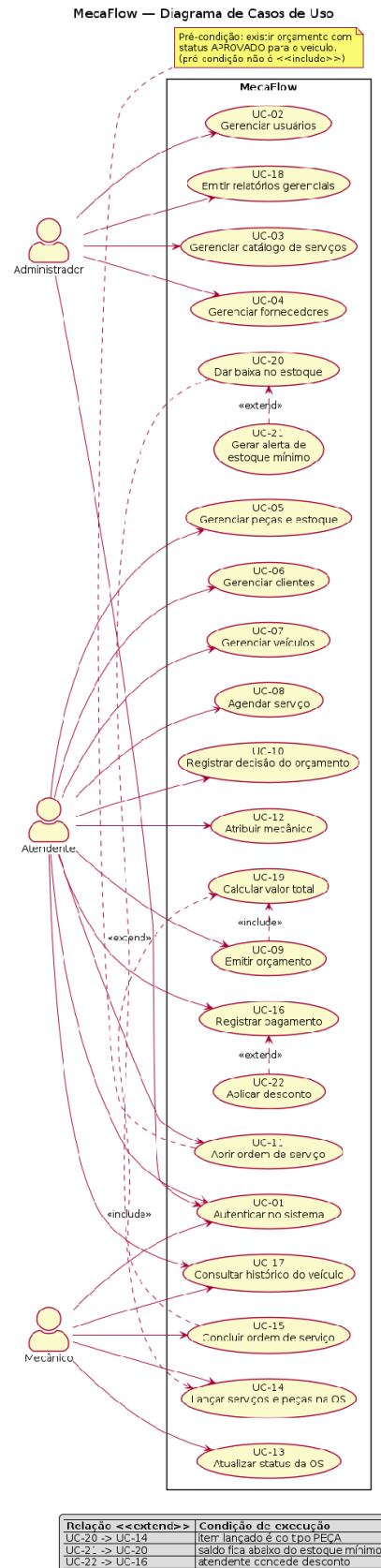


Figura 1. Diagrama de casos de uso do MecaFlow.

As histórias de usuário abaixo resumem o valor que cada ator espera do sistema:

- HU-01: Como atendente, quero cadastrar o cliente e seus veículos uma única vez para reaproveitar os dados em todos os atendimentos (UC-06, UC-07).
- HU-02: Como atendente, quero emitir um orçamento com itens de serviço e peça para o cliente decidir com o valor fechado (UC-09).
- HU-03: Como atendente, quero abrir a ordem de serviço a partir de um orçamento aprovado para garantir que só serviço autorizado entre na oficina (UC-10, UC-11).
- HU-04: Como mecânico, quero atualizar o status da OS e lançar o que consumi para o atendente acompanhar o reparo sem me interromper (UC-13, UC-14).
- HU-05: Como administrador, quero relatórios de faturamento e produtividade para decidir preço e escala da equipe (UC-18).
- HU-06: Como atendente, quero que cada peça lançada debite o estoque para o saldo do sistema refletir a prateleira (UC-20, UC-21).

2.3 Diagrama de Sequência do Sistema e Contrato de Operações

Os diagramas de sequência do sistema (DSS) abaixo tratam o MecaFlow como caixa-preta e mostram a troca de mensagens entre o ator e o sistema para três casos de uso centrais: UC-09 (Emitir orçamento), UC-11 (Abrir ordem de serviço) e UC-16 (Registrar pagamento). Em seguida, os contratos descrevem as operações de sistema relevantes no formato Operação, Referências cruzadas, Pré-condições e Pós-condições.

DSS-01 — Diagrama de Sequência do Sistema: Emitir Orçamento (UC-09)

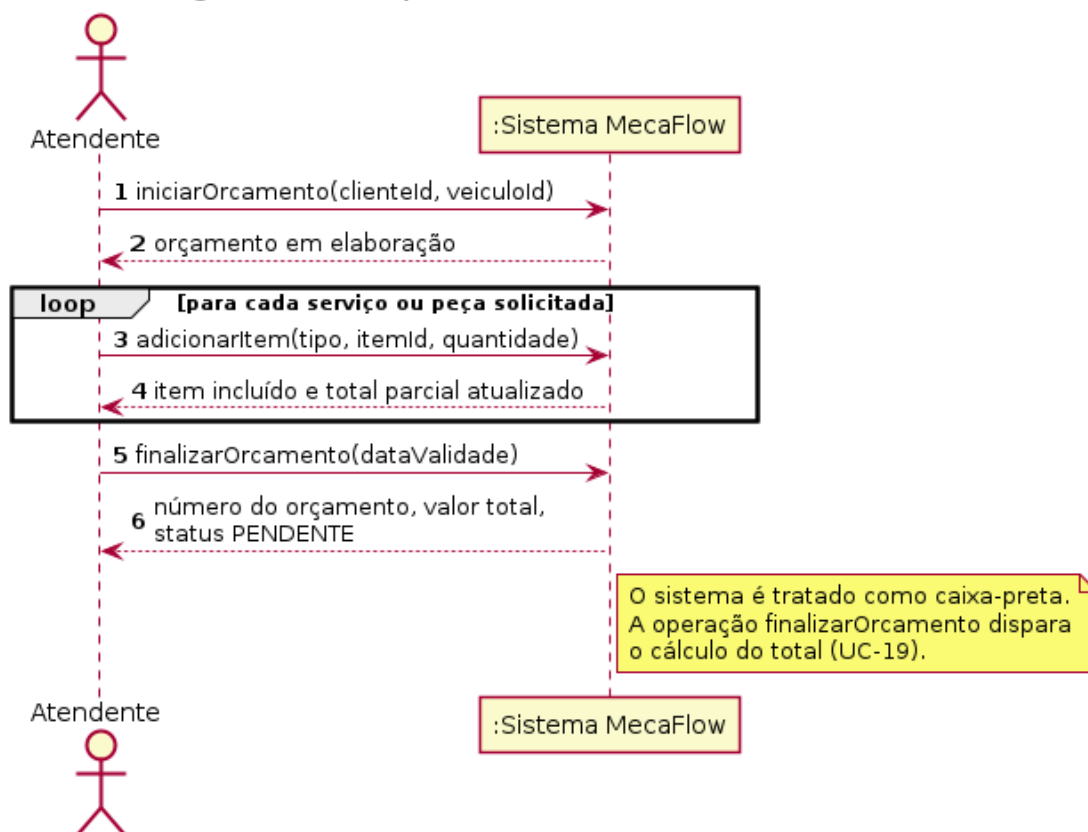


Figura 2. DSS do caso de uso UC-09 Emitir orçamento.

DSS-02 — Diagrama de Sequência do Sistema: Abrir Ordem de Serviço (UC-11)

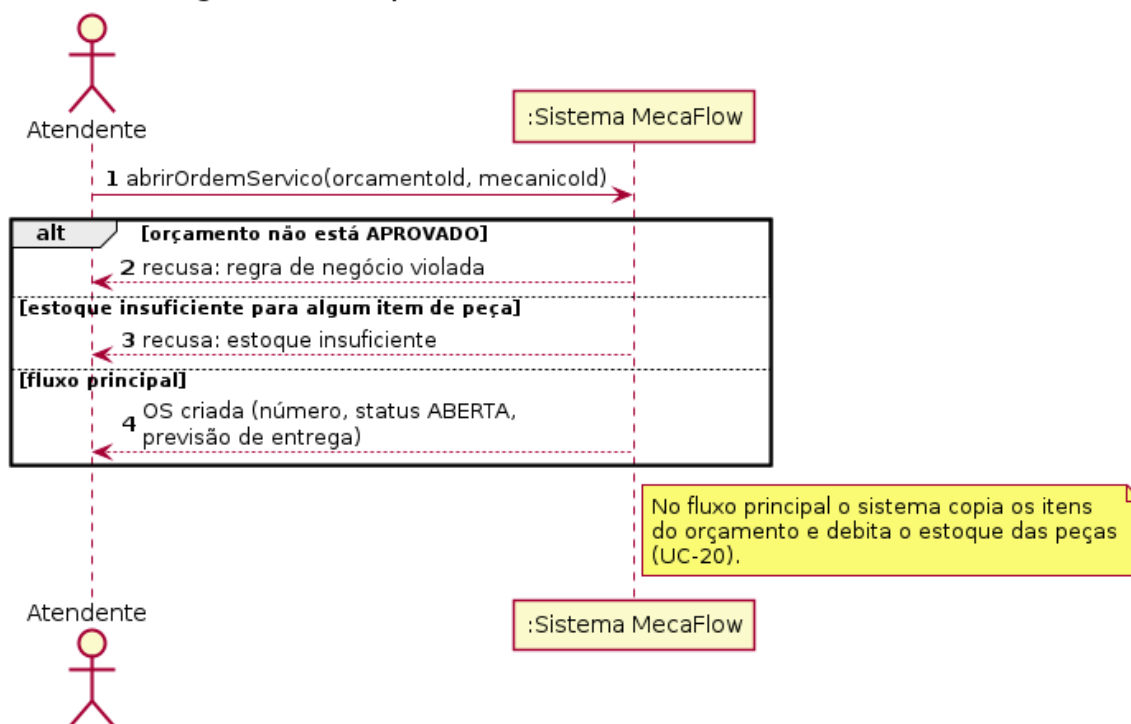


Figura 3. DSS do caso de uso UC-11 Abrir ordem de serviço.

DSS-03 — Diagrama de Sequência do Sistema: Registrar Pagamento (UC-16)

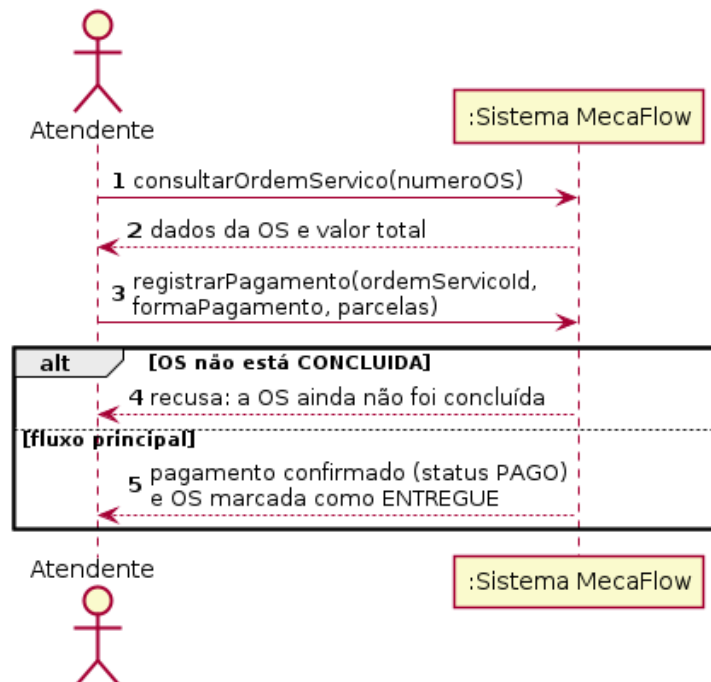


Figura 4. DSS do caso de uso UC-16 Registrar pagamento.

Contratos de operação:

Contrato	CO-01: confirmarEmissao
----------	-------------------------

Operação	confirmarEmissao(dataValidade)
Referências cruzadas	UC-09 Emitir orçamento; Figura 2.
Pré-condições	Existe um orçamento em elaboração com pelo menos um item; o cliente e o veículo estão cadastrados.
Pós-condições	Uma instância de Orcamento foi criada e persistida com status PENDENTE; o atributo valorTotal foi calculado como a soma dos itens (UC-19); o número do orçamento foi gerado; a dataValidade foi registrada; as associações com Cliente e Veiculo foram estabelecidas.

Contrato	CO-02: registrarDecisaoOrcamento
Operação	registrarDecisaoOrcamento(orcamentold, decisao)
Referências cruzadas	UC-10 Registrar decisão do orçamento.
Pré-condições	O orçamento existe com status PENDENTE e está dentro do prazo de validade.
Pós-condições	O atributo status do Orcamento foi alterado para APROVADO ou REJEITADO, conforme a decisão informada pelo cliente; a data da decisão foi registrada.

Contrato	CO-03: abrirOrdemServico
Operação	abrirOrdemServico(orcamentold, mecanicold, kmEntrada)
Referências cruzadas	UC-11 Abrir ordem de serviço; UC-20 Dar baixa no estoque; Figura 3.
Pré-condições	O orçamento está APROVADO e ainda não originou uma OS; o mecânico está ativo; para cada item de peça, quantidadeEstoque é maior ou igual à quantidade solicitada.
Pós-condições	Uma instância de OrdemServico foi criada com status ABERTA e número gerado; os itens do orçamento foram copiados para a OS; o atributo quantidadeEstoque de cada Peca foi decrementado na quantidade lançada; as associações com Orcamento, Cliente, Veiculo e Mecanico foram estabelecidas.

Contrato	CO-04: registrarPagamento
Operação	registrarPagamento(osld, formaPagamento, parcelas)
Referências cruzadas	UC-16 Registrar pagamento; UC-22 Aplicar desconto; Figura 4.
Pré-condições	A ordem de serviço está com status CONCLUIDA e não possui pagamento com status PAGO vinculado.

Pós-condições	Uma instância de Pagamento foi criada com status PAGO, valor igual ao valorTotal da OS (com desconto, quando concedido) e forma de pagamento registrada; a associação com a OrdemServico foi estabelecida; a OS ficou apta à transição para ENTREGUE.
----------------------	---

3. Modelos de Projeto

3.1 Arquitetura

O MecaFlow adota uma arquitetura de monólito modular em camadas. A API REST em Spring Boot segue o padrão MVC com Service Layer e Repository, e o front-end React consome essa API por HTTPS/JSON.

As camadas do back-end têm responsabilidades fixas. O Controller recebe as requisições HTTP, valida o formato de entrada e delega para o serviço, sem regra de negócio. O Service concentra as regras do domínio (abertura de OS, aprovação de orçamento, baixa de estoque), controla as transações e orquestra os repositórios. O Repository, com Spring Data JPA, abstrai o acesso ao PostgreSQL. DTOs e Mappers isolam as entidades das respostas da API. Um filtro de segurança valida o token JWT antes de cada requisição chegar ao Controller, e um Exception Handler global padroniza as respostas de erro.

A equipe escolheu o monólito modular por três razões: o escopo é um sistema de gestão único, a equipe é pequena e o deploy precisa ser simples. Microserviços trariam custo operacional sem benefício neste porte. Os módulos (clientes, veículos, orçamentos, ordens de serviço, estoque) ficam isolados por pacote, o que permite extrair um serviço no futuro se o sistema crescer. Como trade-off, o monólito compartilha um único banco e um único processo: uma falha grave derruba o sistema inteiro e a escala é vertical. Para o porte de uma oficina, a simplicidade compensa essas limitações.

A Figura 5 resume a arquitetura em nível de contêineres: o navegador executa a SPA React, que consome a API Spring Boot por HTTPS/JSON, e a API persiste os dados no PostgreSQL.

MecaFlow — Visão de Arquitetura (nível de contêineres, estilo C4)

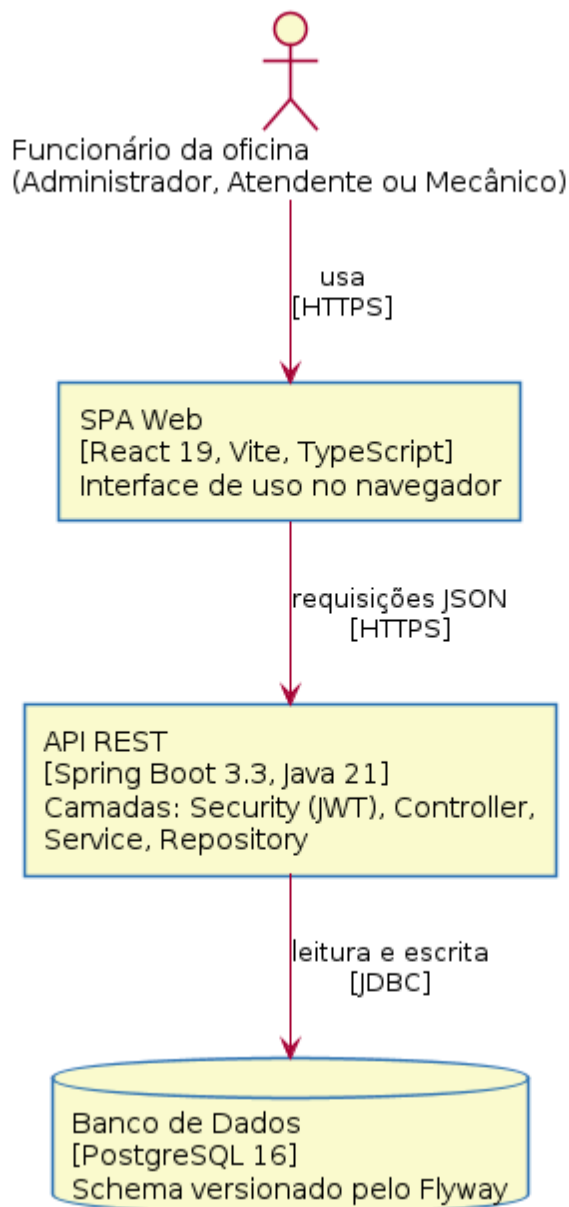


Figura 5. Visão de contêineres do MecaFlow (estilo C4).

O diagrama de componentes da Seção 3.2 (Figura 6) materializa essa visão em camadas com a notação UML.

3.2 Diagrama de Componentes e Implantação

A Figura 6 mostra os componentes do sistema e as dependências entre eles: a SPA React no navegador, os componentes internos da API (segurança, controllers, DTOs, serviços, repositórios e tratamento de exceções), o banco PostgreSQL e o Flyway como responsável pelo versionamento do schema.

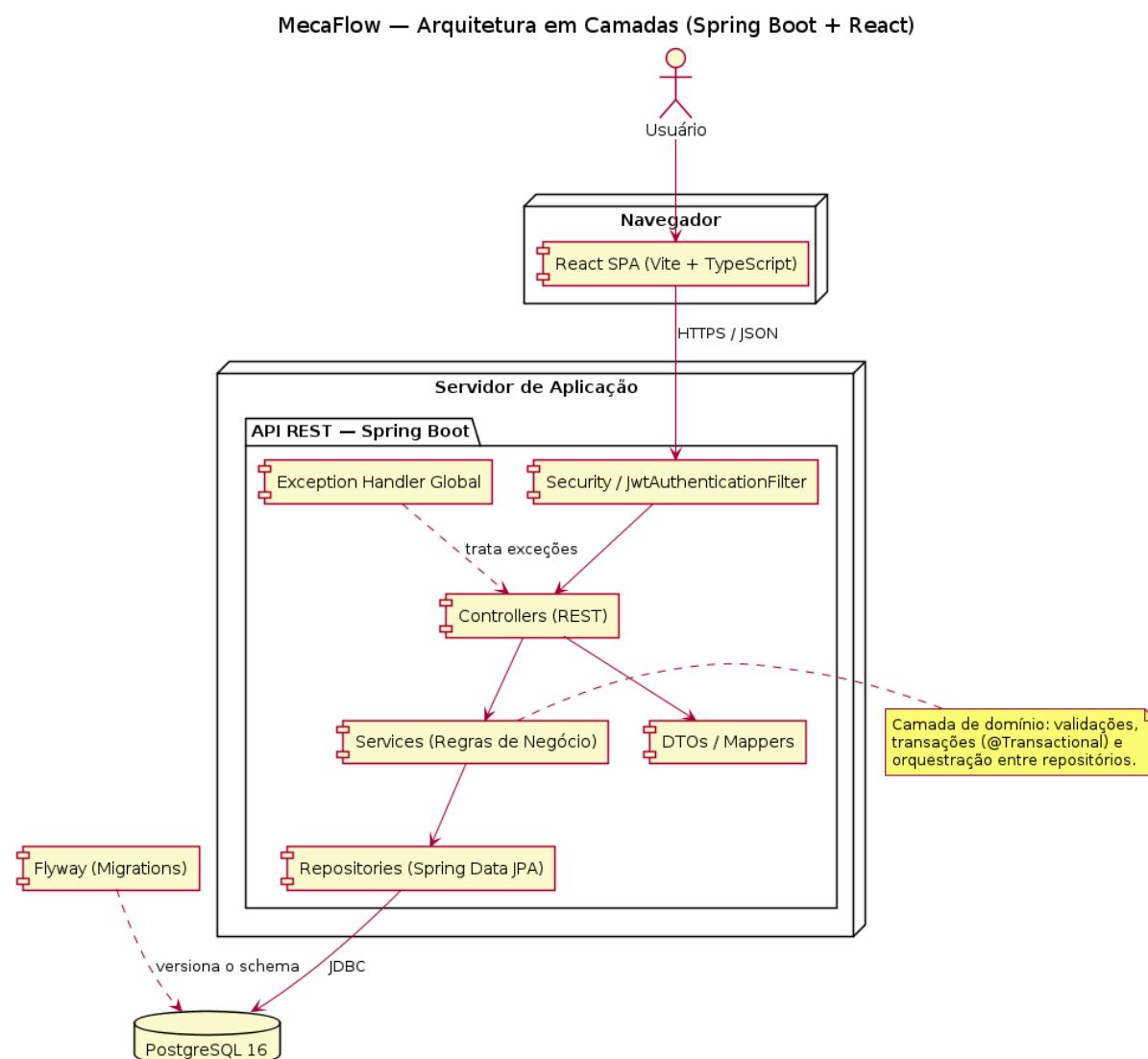


Figura 6. Diagrama de componentes do MecaFlow.

A Figura 7 mostra a alocação dos componentes para execução: o build estático do React servido pela Vercel, a API Spring Boot e o PostgreSQL em containers no Railway, e o navegador do usuário consumindo o front por HTTPS.

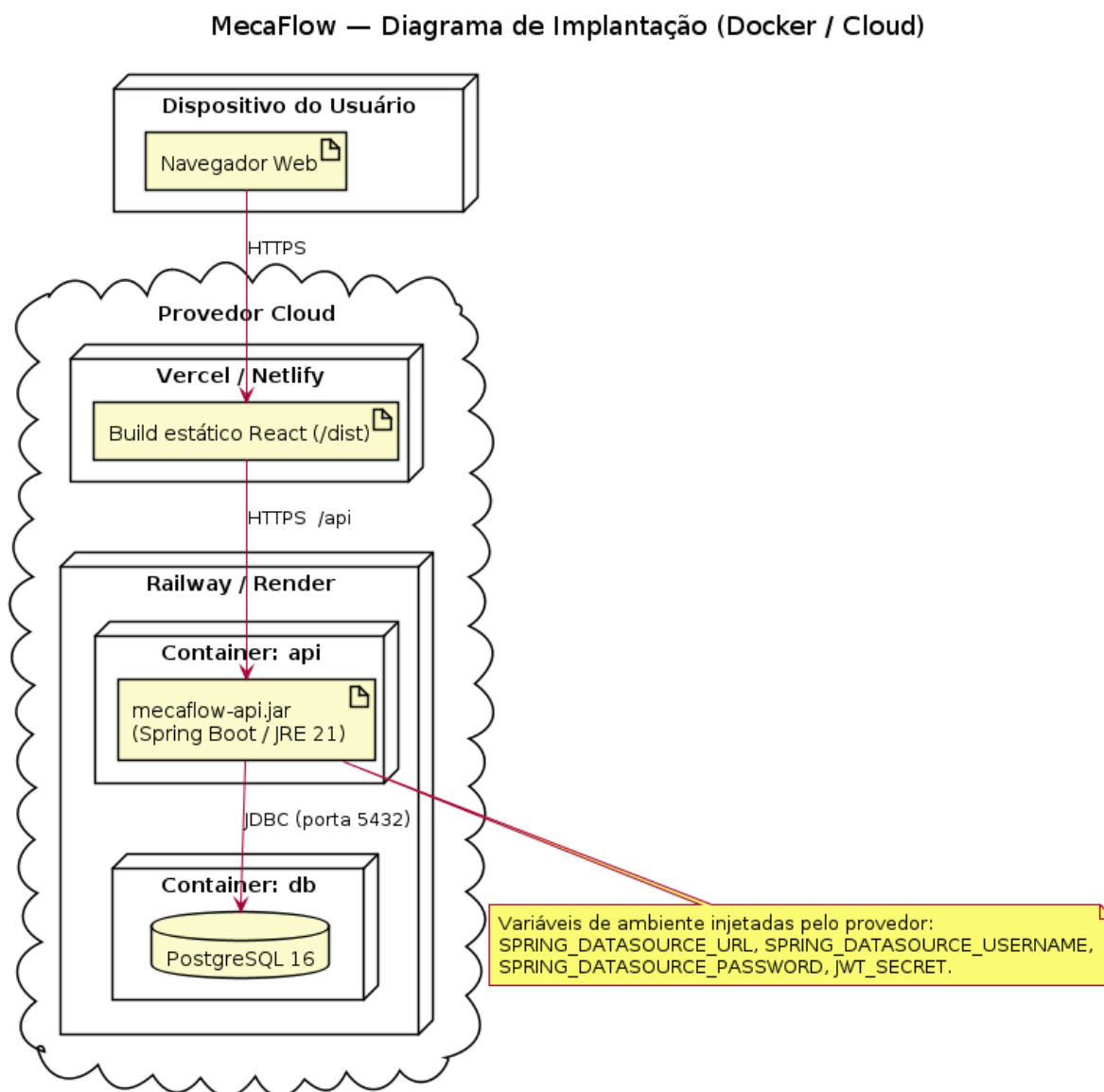


Figura 7. Diagrama de implantação (Docker/Cloud).

3.3 Diagrama de Classes

A Figura 8 apresenta o modelo de domínio com 13 entidades e 7 enumerações. O ciclo de negócio aparece nas associações: Cliente possui Veículos; Agendamento, Orcamento e OrdemServico referenciam cliente e veículo; um Orcamento aprovado origina no máximo uma OrdemServico; os itens de orçamento e de OS apontam para Servico ou Peca; Fornecedor fornece Pecas; Pagamento quita a OS. As operações de negócio ficam nas entidades que guardam o estado, como Peca.baixarEstoque(), Orcamento.aprovar() e OrdemServico.concluir().

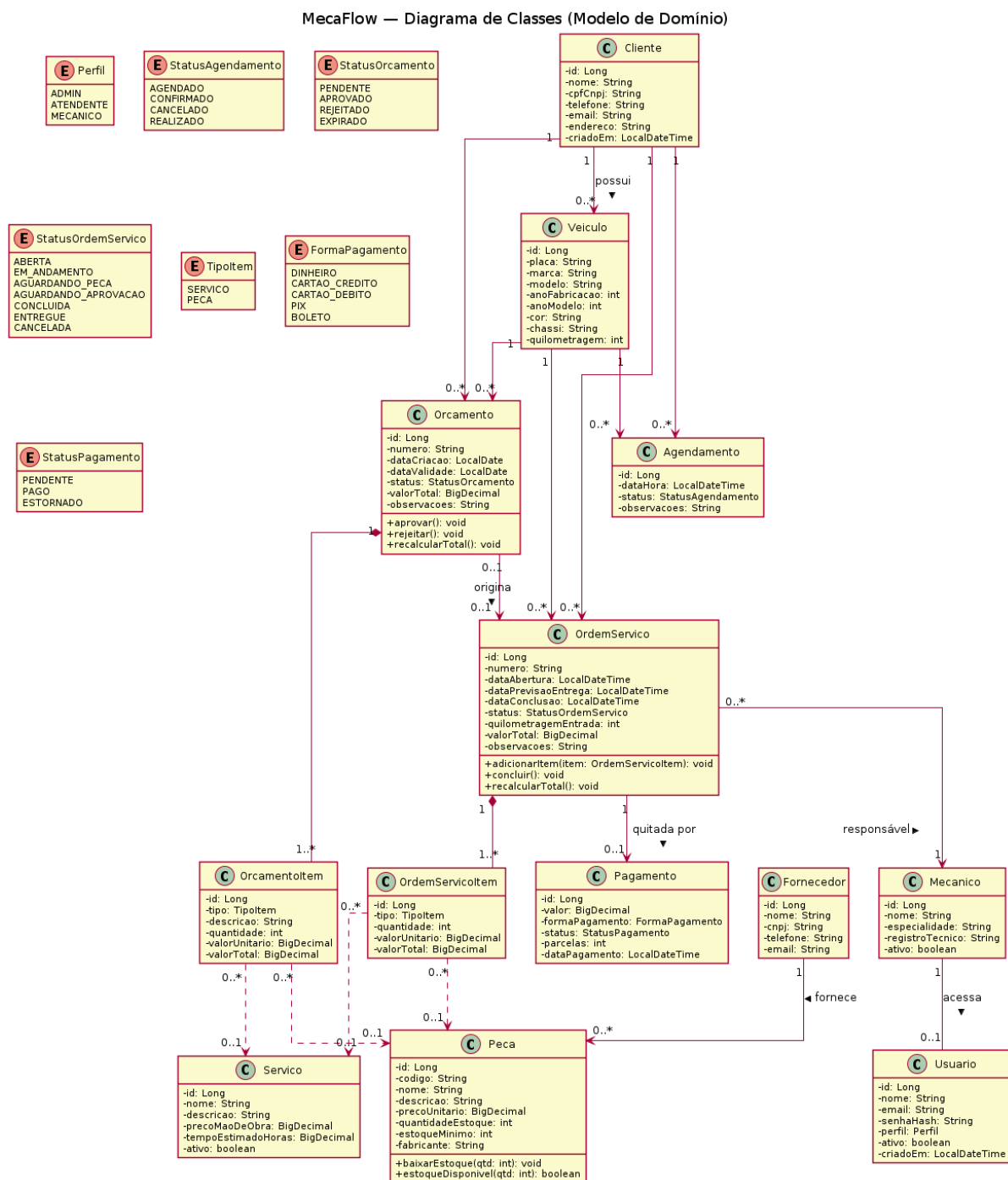


Figura 8. Diagrama de classes (modelo de domínio).

3.4 Diagramas de Sequência

Os diagramas de sequência abaixo detalham a realização de dois casos de uso na arquitetura em camadas. A Figura 9 cobre a autenticação (UC-01): o AuthController delega ao AuthService, que autentica as credenciais via AuthenticationManager e repositório e, em caso de sucesso, gera o token JWT; nas requisições seguintes, o filtro JWT valida o token antes do Controller. A Figura 10 cobre a abertura de OS (UC-11): o serviço valida o status do orçamento, reserva cada peça com baixa de estoque (UC-20) e persiste a OS com status ABERTA; estoque insuficiente ou orçamento não aprovado interrompem o fluxo com erro.

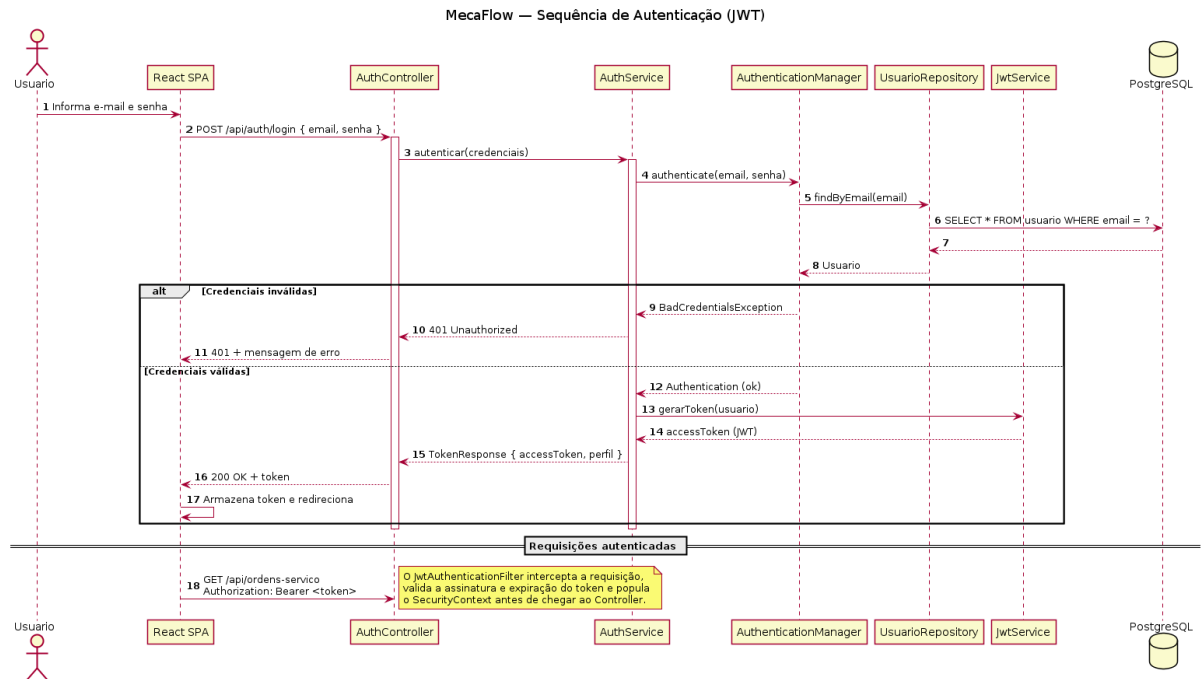


Figura 9. Diagrama de sequência da autenticação com JWT (UC-01).

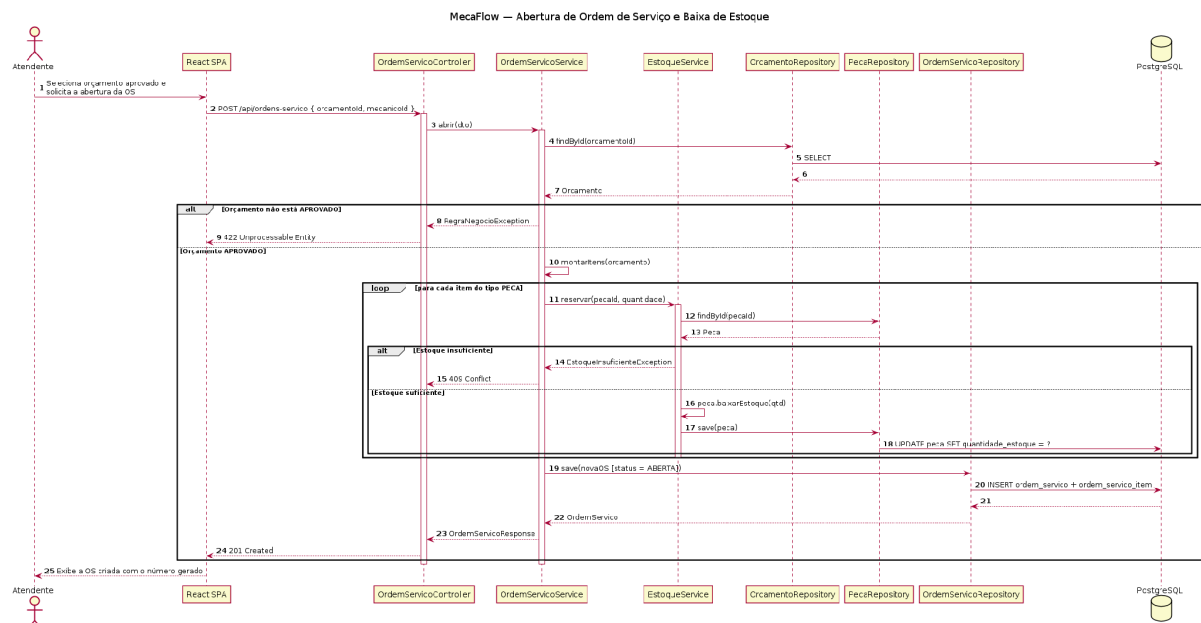


Figura 10. Diagrama de sequência da abertura de ordem de serviço (UC-11).

3.5 Diagramas de Comunicação

A Figura 11 apresenta a realização do UC-11 na notação de comunicação, com a numeração das mensagens indicando a ordem das interações entre os objetos: o controller recebe a requisição (1), o serviço carrega e valida o orçamento (1.1 a 1.3), reserva as peças com baixa de estoque (1.4, iterando sobre os itens) e persiste a nova OS (1.5).

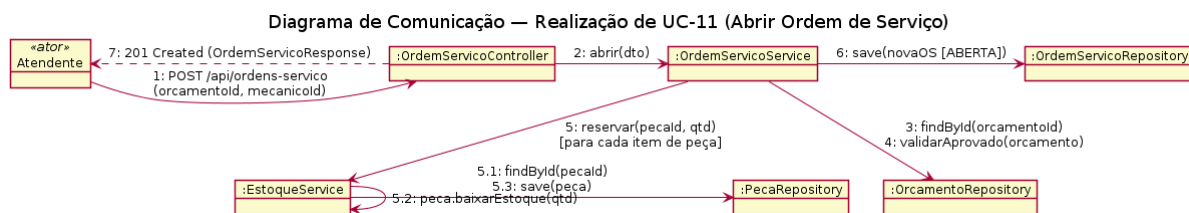


Figura 11. Diagrama de comunicação da abertura de ordem de serviço (UC-11).

A Figura 12 apresenta a realização do UC-16 (registrar pagamento) na notação de comunicação: o controller recebe a requisição (1), o serviço carrega a OS e valida o status (1.1 a 1.3), registra o pagamento (1.4) e atualiza a OS para ENTREGUE quando a quitação se confirma (1.5).



Figura 12. Diagrama de comunicação do registro de pagamento (UC-16).

3.6 Diagramas de Estados

A Figura 13 modela o ciclo de vida da Ordem de Serviço. A OS nasce ABERTA a partir de um orçamento aprovado, entra EM_ANDAMENTO quando o mecânico inicia o reparo e pode aguardar peça ou aprovação de item extra. A conclusão recalcula o valor total, e a entrega exige pagamento com status PAGO. O cancelamento estorna o estoque das peças reservadas.

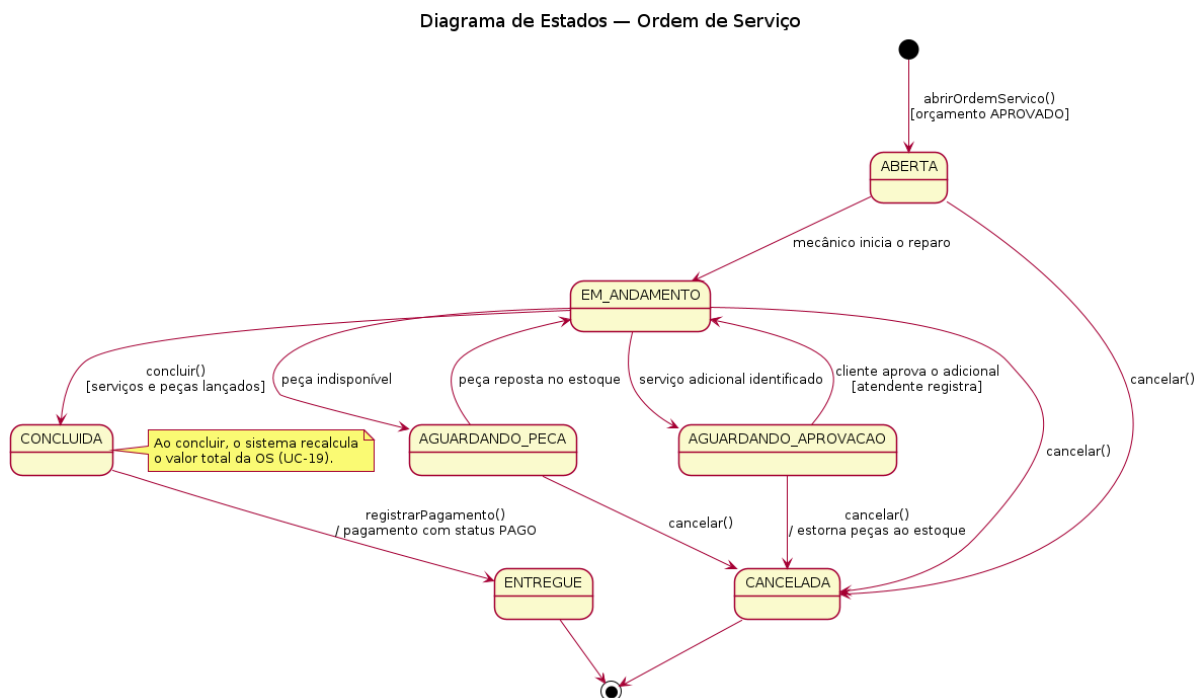


Figura 13. Diagrama de estados da Ordem de Serviço.

A Figura 14 modela o ciclo de vida do Orçamento: nasce PENDENTE na emissão e termina APROVADO, REJEITADO ou EXPIRADO. O estado APROVADO habilita a abertura da OS (UC-11).

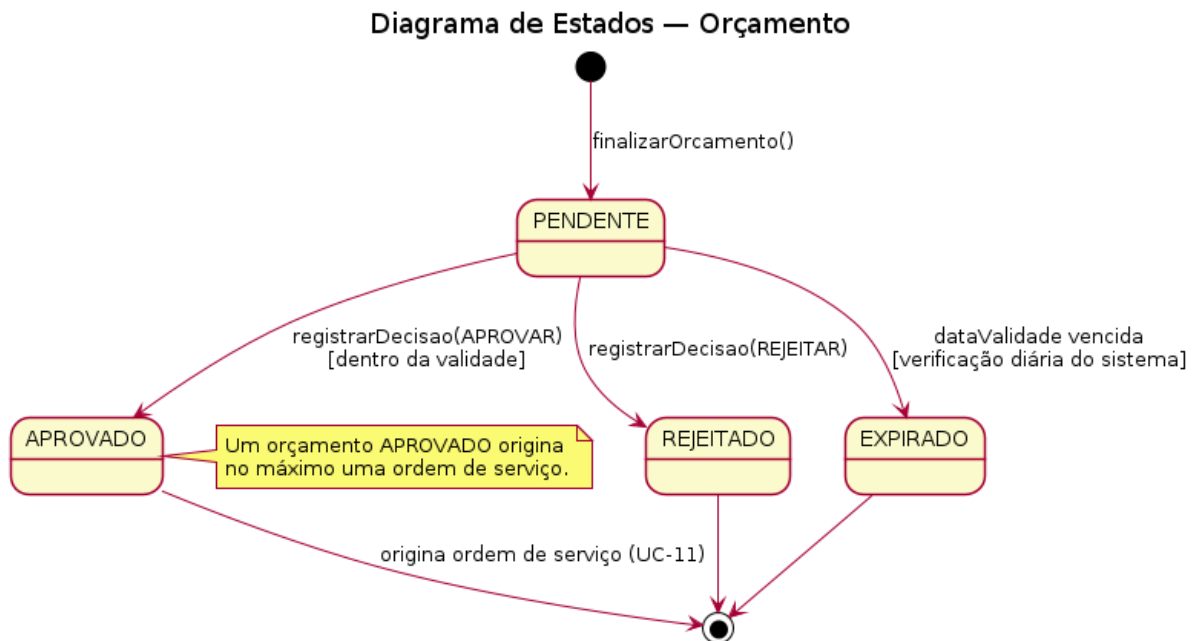


Figura 14. Diagrama de estados do Orçamento.

4. Modelos de Dados

A Figura 15 apresenta o modelo entidade-relacionamento físico para PostgreSQL 16. O modelo usa nomes de tabela e coluna em snake_case, chaves primárias BIGSERIAL, chaves estrangeiras com restrição de integridade referencial e colunas únicas para os identificadores de negócio (e-mail do usuário, CPF/CNPJ do cliente, placa do veículo, código da peça e números de orçamento e de OS). Valores monetários usam NUMERIC(10,2). O Flyway versiona o schema por migrações SQL aplicadas na inicialização da aplicação.

Duas decisões de integridade merecem registro: a tabela pagamento referencia ordem_servico com chave estrangeira única, o que garante no máximo um pagamento por OS; e as tabelas de item (orcamento_item e ordem_servico_item) guardam o valor unitário praticado no momento do lançamento, o que preserva o histórico mesmo quando o preço de catálogo muda.

MecaFlow — Modelo Entidade-Relacionamento (PostgreSQL)

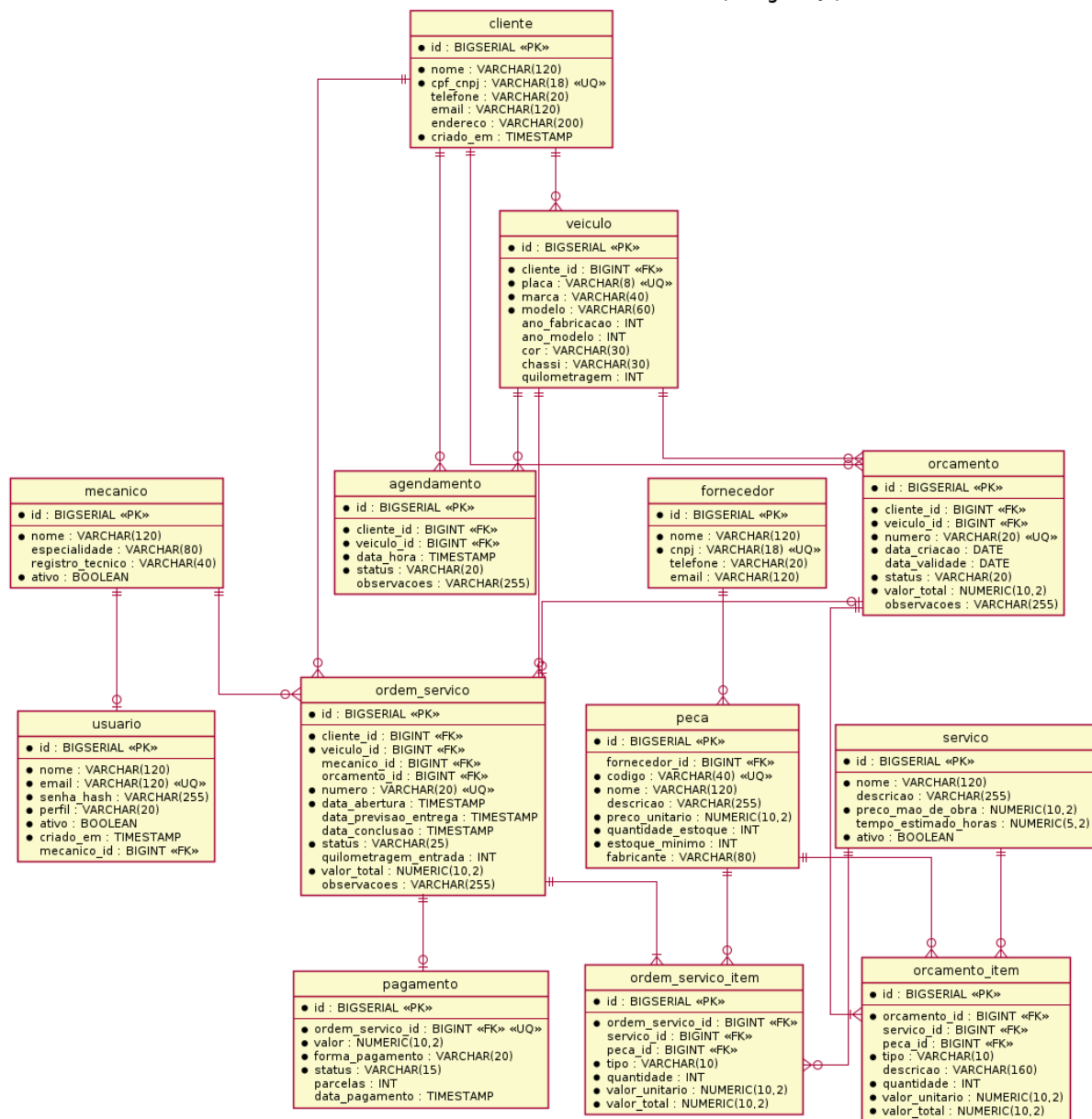


Figura 15. Modelo entidade-relacionamento (PostgreSQL).

4.1 Estratégia de Mapeamento Objeto-Relacional

O back-end mapeia as entidades de domínio para as tabelas do PostgreSQL com JPA/Hibernate. A tabela abaixo resume as principais decisões de mapeamento adotadas.

Decisão de mapeamento	Implementação
Identificadores	Chave primária com @Id e @GeneratedValue(strategy = IDENTITY), mapeada para BIGSERIAL no PostgreSQL.
Associações um-para-muitos	@OneToMany no lado pai (Orcamento e OrdemServico para seus itens) com cascade = ALL e orphanRemoval = true, de modo que os itens seguem o ciclo de vida do dono.

Associações muitos-para-um	@ManyToOne com @JoinColumn nas pontas que carregam a chave estrangeira (item para Servico ou Peca, Veiculo para Cliente, OrdemServico para Orcamento), com busca LAZY.
Associações um-para-um	@OneToOne entre Pagamento e OrdemServico e entre Mecanico e Usuario, com chave estrangeira única que impede duplicidade.
Enumerações	@Enumerated(EnumType.STRING) para persistir o nome do valor (status e formas de pagamento) em vez do índice, o que mantém o dado legível e estável.
Versionamento do schema	O Flyway aplica as migrações SQL versionadas (V1__init.sql e seguintes) na inicialização, sem geração automática de DDL pelo Hibernate em produção (ddl-auto = validate).

Tabela 1. Estratégia de mapeamento objeto-relacional.